

Task 3: File System Operations and Security

User Guide and Security Analysis — ST5004CEM Operating Systems and Security

Part A — User Guide

1. Creating a user

```
User alice(1, "alice", "password123");
```

The constructor immediately hashes the supplied password with `std::hash<std::string>` and stores only the hash. The plain-text password is never retained in memory beyond the constructor call.

2. Authenticating

```
bool ok = alice.authenticate("password123"); // true
```

`authenticate()` hashes the supplied password and compares it against the stored hash; the two plain-text strings are never compared directly.

3. Creating, writing, reading and deleting files

```
fs.createFile(alice, "notes.txt");  
fs.writeFile(alice, "notes.txt", "some content");  
bool allowed; std::string data = fs.readFile(bob, "notes.txt", allowed);  
fs.deleteFile(alice, "notes.txt");
```

Every one of these calls checks the calling user's permissions before acting and records the outcome: allowed or denied, in the audit log, whether or not the action succeeds.

4. Permission model

Each file tracks four flags: `ownerCanRead`, `ownerCanWrite`, `othersCanRead` and `othersCanWrite`. By default a newly created file grants its owner full read/write access and grants everyone else no access at all, until explicitly changed.

5. Encryption

```
file->encrypt(); // stores the data XOR-ed with a fixed key  
file->decrypt(); // XOR-ing again with the same key restores it
```

`readFile()` automatically decrypts a copy of the data to return to the caller, then re-encrypts the stored copy so the file is never left decrypted at rest.

6. Viewing the audit trail

```
fs.printAuditLog();
```

Prints every CREATE/READ/WRITE/DELETE attempt made so far, each with a timestamp, the username, the file name, and whether it was ALLOWED or DENIED.

Part B — Security Analysis

Password storage

Passwords are hashed rather than stored in plain text, so a memory dump or accidental print of a User object does not directly reveal a password. However, `std::hash<std::string>` is a fast, general-purpose hash intended for hash tables, not a cryptographic hash: it has no salt, so two users with the same password produce the same hash, and it is fast enough to make brute-force or rainbow-table attacks practical. A production system should use a slow, salted algorithm designed for passwords, such as bcrypt or Argon2.

Encryption

File contents can be encrypted with a single-byte XOR cipher. This correctly hides data from a casual viewer and demonstrates the encrypt/decrypt requirement, but it is cryptographically weak: XOR with a single repeating byte is broken almost immediately by frequency analysis, and is trivial to reverse if even one byte of plaintext is known. A production system would use an authenticated cipher such as AES-GCM with a securely generated, per-file key.

Access control

The permission model distinguishes only 'owner' and 'others' rather than the fuller owner/group/others model used by real operating systems, and there is currently no way to grant access to a specific second user without granting it to everyone. This was a deliberate simplification for clarity; a fuller implementation would add group membership or per-user access control lists.

Audit logging

Every access attempt (successful or denied) is logged with a timestamp, demonstrated directly in testing: when user bob attempted to read a file owned by alice, the system produced the log line 'user=bob action=READ file=alice_notes.txt result=DENIED', while alice's own read of the same file produced 'result=ALLOWED'. This gives a verifiable, timestamped trail of who accessed what. The current limitation is that the log exists only in memory for the lifetime of the program and has no protection against tampering; a production system would append to a write-once log file or an external, access-controlled logging service.

Summary

The system meets the brief's core security requirements: authentication, permissions, encryption and audit logging. And each mechanism was tested and produced the expected allow/deny behaviour. The limitations documented above (non-cryptographic password hashing, weak encryption, coarse permission granularity, and an unprotected in-memory log) are named deliberately: recognising a control's limits is itself part of a sound security analysis, rather than a weakness to conceal.